Adquisición de Datos de Corriente Utilizando Resistencia Shunt

Ingeniería Mecánica y Electromecánica

Poma Carita Gary Javier
poma.9163997@gmail.com

Docente: Ing. Vidher W. Bustillos D.

Resumen

El presente trabajo describe el diseño e implementación de un sistema de adquisición de datos de corriente utilizando un microcontrolador Arduino y una resistencia shunt como sensor. El objetivo fue medir la corriente consumida por un motor DC, convertirla en una señal de voltaje proporcional mediante la resistencia shunt, digitalizarla con el convertidor analógico-digital (ADC) del Arduino y almacenar los datos en formato *.CSV para su posterior análisis en Python. Los resultados obtenidos muestran variaciones coherentes de la corriente en función del comportamiento del motor, validando el correcto funcionamiento del sistema. Se concluye que el uso de resistencias shunt constituye una técnica eficiente y económica para la medición de corriente en sistemas electrónicos.

Palabras claves: adquisición de datos, Arduino, corriente, ADC.

Current data acquisition with Shunt Resistor

Abstract

This paper presents the design and implementation of a current data acquisition system using an Arduino microcontroller and a shunt resistor as sensing element. The objective was to measure the current consumed by a DC motor, convert it into a proportional voltage signal, digitize it through the ADC module, and store the data in *.CSV format for later analysis in Python. The obtained results showed consistent current variations according to motor operation. The integration of hardware and software demonstrates an effective solution for real-time monitoring systems.

Keywords: *data acquisition, Arduino, current, shunt resistor, ADC.*

Introducción

La adquisición de datos es fundamental en la ingeniería, ya que permite monitorear y analizar variables físicas en tiempo real. En aplicaciones eléctricas, la medición de corriente es esencial para evaluar el comportamiento de cargas como motores eléctricos.

Una técnica ampliamente utilizada para medir corriente es el uso de una **resistencia shunt**, la cual convierte la corriente en una caída de voltaje proporcional según la ley de Ohm. Este voltaje puede ser medido por un microcontrolador como Arduino mediante su convertidor analógico-digital (ADC).

ADQUISICIÓN DE DATOS DE CORRIENTE CON RESISTENCIA SHUNT

El objetivo de esta práctica fue implementar un sistema de adquisición, almacenamiento y visualización de datos de corriente utilizando Arduino y Python.

Desarrollo de la Investigación

Solución propuesta

El sistema implementado se basa en una resistencia shunt conectada en serie con un motor DC. La corriente que circula por el circuito genera una caída de voltaje en la resistencia, la cual es medida por el pin analógico A0 del Arduino.
La relación entre corriente y voltaje está dada por la ley de Ohm:

$$I = \frac{V_{shunt}}{R_{shunt}} \quad \ldots \text{(ec.1)}$$

Donde:

- $I: corriente\ (A)$
- $V_{shunt}: voltaje\ en\ la\ resistencia\ shunt\ (V)$
- $R_{shunt}: resistencia\ shunt\ (\Omega)$

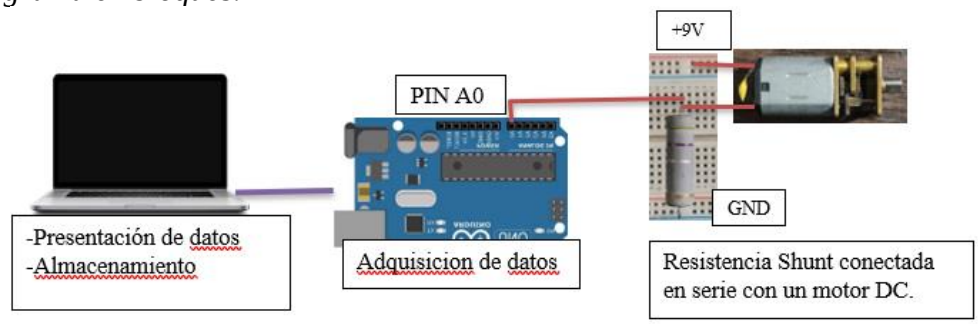
El Arduino digitaliza esta señal y la envía al computador, donde Python procesa, grafica y almacena los datos.

Requerimientos materiales y equipos

- Sistema micro programable Arduino UNO
- Resistencia Shunt
- Motor DC
- Fuente de Alimentacion (9V)
- Protoboard y cables de conexión
- Computadora personal
- Software Arduino IDE
- Python (PySerial, Matplotlib)

Diagrama en bloques

Figura 1. *Diagrama en bloques.*



Fuente: elaboración propia.

Codificación de los programas

IDE Arduino:

Figura 2. *Codificación en el IDE Arduino.*

ad_datos_corriente1_Shunt.ino

```
#define VREF_MAX        5.0
#define RESOLUCION      1023.0
int   sensor_shunt = A0;
float voltajeShunt, corrienteMotor = 0.0;

void setup() {
    pinMode(sensor_shunt, INPUT);
    Serial.begin(9600);
}

void loop() {
    voltajeShunt = analogRead(sensor_shunt) * (VREF_MAX / RESOLUCION);
    corrienteMotor = (voltajeShunt / 0.5) * 1000;
    Serial.print("Corriente Motor: ");
    Serial.print(corrienteMotor);
    Serial.print("mA - Voltaje Shunt: ");
    Serial.print(voltajeShunt);
    Serial.println(" Volts");
    delay(500);
}
```

Fuente: elaboración propia.

Codificación en Python:

Figura 3. *Codificación en Python.*

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time

# Conexión al Arduino
arduino = serial.Serial(port='COM4', baudrate=9600, timeout=1)

# Archivo CSV
archivo = open("AD_DATOS_DE_CORRIENTE.csv", "w", newline="", encoding="utf-8")
escritor = csv.writer(archivo, delimiter=';')

escritor.writerow(["REGISTRO DE DATOS - RESISTENCIA SHUNT"])
escritor.writerow(["Fecha:", time.strftime("%d/%m/%Y")])
escritor.writerow(["Hora inicio:", time.strftime("%H:%M:%S")])
escritor.writerow([])
escritor.writerow(["Tiempo (s)", "Corriente (A)"])

# Variables
tiempo = []
valores = []
inicio = time.time()
pausado = False

# Figura
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.2)

linea, = ax.plot([], [], lw=2)

ax.set_title("Adquisicion de datos de corriente")
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Corriente (mA)")
```

```
ax.set_ylim(0, 100)

# Botón pausa
def toggle_pausa(event):
    global pausado
    pausado = not pausado
    if pausado:
        boton.label.set_text("Reanudar")
    else:
        boton.label.set_text("Pausar")

ax_boton = plt.axes([0.8, 0.05, 0.1, 0.075])
boton = Button(ax_boton, "Pausar")
boton.on_clicked(toggle_pausa)

# Inicializar
def init():
    linea.set_data([], [])
    return linea,

# Actualizar gráfica
def actualizar(frame):
    global pausado

    if pausado:
        return linea,
    dato = arduino.readline().decode('utf-8').strip()

    if dato:

        try:
            # Extraer corriente desde el texto enviado por Arduino
            if "Corriente Motor:" in dato:

                corriente_mA = float(dato.split("Corriente Motor:")[1].split("mA")[0])
                corriente = corriente_mA / 1000   # convertir a Amperios

                tiempo_actual = round(time.time() - inicio, 2)

                tiempo.append(tiempo_actual)
                valores.append(round(corriente_mA,3))

                escritor.writerow([tiempo_actual, round(corriente_mA,3)])
                archivo.flush()

                if len(tiempo) > 100:
                    tiempo.pop(0)
                    valores.pop(0)

                linea.set_data(tiempo, valores)
                ax.set_xlim(max(0, tiempo[-1]-10), tiempo[-1])
        except:
            pass
    return linea,
# Animación
ani = animation.FuncAnimation(fig, actualizar, init_func=init, blit=True, interval=100)

plt.show()
archivo.close()
arduino.close()
```

Fuente: elaboración propia.

El programa permite:

- Leer datos desde el puerto serial
- Graficar en tiempo real
- Guardar datos en archivo CSV
- Pausar/reanudar la adquisición de datos en la grafica.

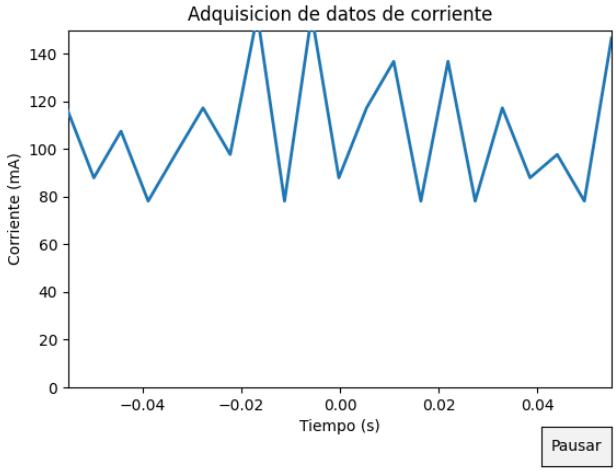
Resultados Obtenidos

Los datos adquiridos fueron almacenados en un archivo CSV con dos columnas: tiempo (s) y corriente (mA). Los valores obtenidos muestran variaciones de corriente conforme al comportamiento dinámico del motor DC.

Estas variaciones se deben principalmente al ruido eléctrico del motor DC, también contribuye la sensibilidad el ADC integrado en el Arduino.

La señal también puede ser inestable debido a un valor bajo en la resistencia Shunt, esto provoca una caída de voltaje muy pequeña por lo tanto cualquier ruido o interferencia afecta mucho la medición.

Figura 4. *Grafica de las Variaciones de Corriente en el motor DC obtenida con Python*



Fuente: elaboración propia.

Tabla 1. *Datos de tiempo y corriente obtenidos del archivo CSV.*

Tiempo (s)	Corriente(mA)
1.63	78.2
2.14	117.3
2.64	117.3
3.14	87.98
3.64	87.98
4.15	117.3
4.65	87.98
5.15	107.53

5.65	78.2
6.15	97.75
6.66	117.3
7.16	97.75
7.66	156.4
8.16	78.2
8.66	156.4
9.16	87.98
9.67	117.3
10.17	136.85
10.67	78.2
11.17	136.85

Fuente: elaboración propia.

Discusión y conclusiones

Los resultados obtenidos demuestran que el sistema implementado permite medir y registrar correctamente la corriente en un sistema físico real. La resistencia shunt resultó ser un método confiable si se usa un filtro electrónico o el modulo INA219, para transformar la corriente en una señal de voltaje medible por el Arduino.

Las fluctuaciones observadas en la señal de corriente se deben principalmente al ruido eléctrico generado por el motor DC, también a la alta sensibilidad del sistema de adquisición basado en Arduino. A diferencia del amperímetro, que presenta un valor promedio estable, el sistema captura variaciones instantáneas de la señal, lo que explica la presencia de picos en la gráfica. La ausencia de filtrado electrónico y el uso de una resistencia shunt de bajo valor también se suman a estas variaciones. En conclusión, la integración de Arduino y Python constituye una herramienta eficaz para la adquisición y análisis de datos en tiempo real.

Referencias bibliográficas

Bustillos D., V. W. (2026). *POO_ADQUISICION_1* [Presentación de PowerPoint]. Facultad de Ingeniería, Universidad Mayor de San Andrés.

midudev. (2025, marzo 25). *Curso COMPLETO de Python DESDE CERO para Principiantes 2025* [Video]. YouTube. https://youtu.be/TkN2i-_4N4g

ANEXOS

Figura 5. *Hoja de datos.*

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

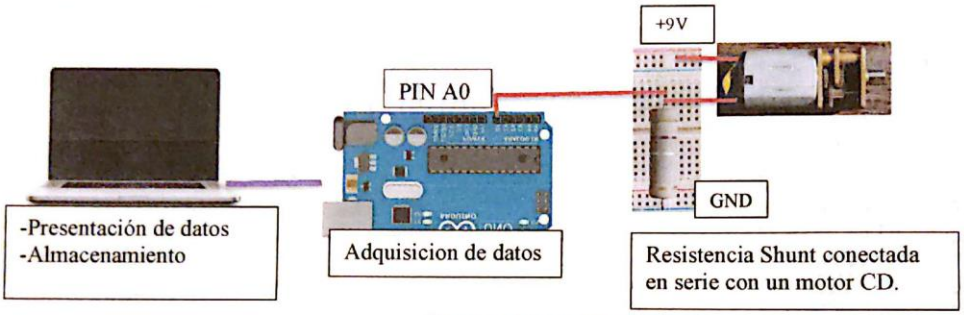
HOJA DE DATOS

LABORATORIO No: 2	FECHA:12/03/2026	Vo.Bo. Ing. Vidher W. Bustillos D.					
Adquisición de Datos de Corriente utilizando resistencia Shunt							
PARTICIPANTES (Apellidos Nombres)		Llenado datos (15)	Presentación de resultados (10)	Funcionamiento Adecuado (30)	Informe en Plataforma (15)	INFORME Físico (30)	NOTA LABORATORIO (100)
1.-Poma Carita Gary Javier		15	10	30			
2.-							

Nombre competo de la Práctica a desarrollar:
Adquisición de Datos de Corriente con Microcontrolador y Resistencia Shunt

Antecedentes del problema a solucionar:
Adquirir datos de un sistema físico real con un microcontrolador, graficar y almacenar los datos adquiridos en Python.

Diagrama en bloques o algoritmo de solución del problema:



Fuente: elaboración propia.

Codificación completa del programa:

```python
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time

# Conexión al Arduino
arduino = serial.Serial(port='COM4', baudrate=9600, timeout=1)

# Archivo CSV
archivo = open("AD_DATOS_DE_CORRIENTE.csv", "w", newline="",
encoding="utf-8")
escritor = csv.writer(archivo, delimiter=';')

escritor.writerow(["REGISTRO DE DATOS - RESISTENCIA SHUNT"])
escritor.writerow(["Fecha:", time.strftime("%d/%m/%Y")])
escritor.writerow(["Hora inicio:", time.strftime("%H:%M:%S")])
escritor.writerow([])
escritor.writerow(["Tiempo (s)", "Corriente (A)"])

# Variables
tiempo = []
valores = []
inicio = time.time()
pausado = False

# Figura
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.2)

linea, = ax.plot([], [], lw=2)

ax.set_title("Adquisicion de datos de corriente")
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Corriente (mA)")
ax.set_ylim(0, 100)
```

```python
# Botón pausa
def toggle_pausa(event):
    global pausado
    pausado = not pausado
    if pausado:
        boton.label.set_text("Reanudar")
    else:
        boton.label.set_text("Pausar")

ax_boton = plt.axes([0.8, 0.05, 0.1, 0.075])
boton = Button(ax_boton, "Pausar")
boton.on_clicked(toggle_pausa)

# Inicializar
def init():
    linea.set_data([], [])
    return linea,

# Actualizar gráfica
def actualizar(frame):

    global pausado

    if pausado:
        return linea,

    dato = arduino.readline().decode('utf-8').strip()

    if dato:

        try:
            # Extraer corriente desde el texto enviado por
Arduino
            if "Corriente Motor:" in dato:
```

Fuente: elaboración propia.

Figura 6. *Hoja de datos.*

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

```
            corriente_mA = float(dato.split("Corriente
Motor:")[1].split("mA")[0])
            corriente = corriente_mA / 1000   #
convertir a Amperios

            tiempo_actual = round(time.time() -
inicio, 2)

            tiempo.append(tiempo_actual)
            valores.append(round(corriente_mA,3))

            escritor.writerow([tiempo_actual,
round(corriente_mA,3)])
            archivo.flush()

            if len(tiempo) > 100:
                tiempo.pop(0)
                valores.pop(0)
```

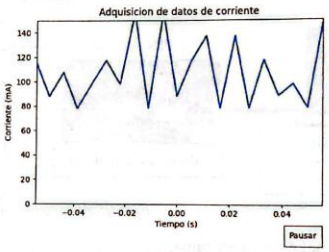
```
            linea.set_data(tiempo, valores)
            ax.set_xlim(max(0, tiempo[-1]-10),
tiempo[-1])

        except:
            pass

    return linea,

# Animación
ani = animation.FuncAnimation(fig, actualizar,
init_func=init, blit=True, interval=100)

plt.show()

archivo.close()
arduino.close()
```

Captura de ventanas del funcionamiento adecuado del algoritmo o codificación del programa

Archivo *csv:

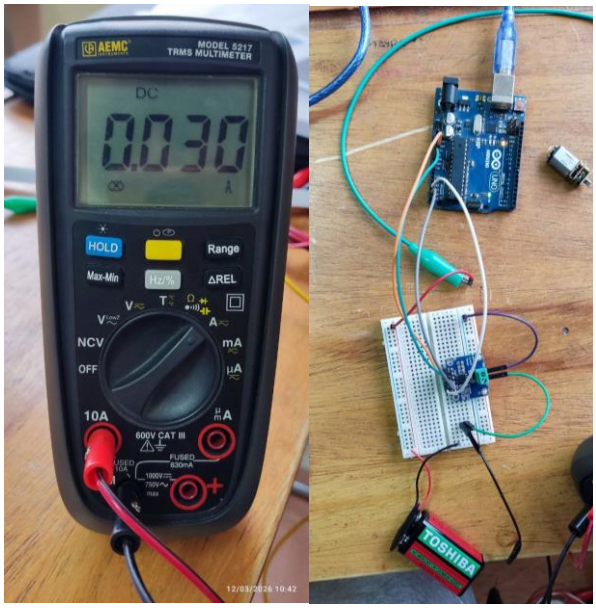
Tiempo (s)	Corriente (A)
1.63	78.2
2.14	117.3
2.64	117.3
3.14	87.98
3.64	87.98
4.15	117.3
4.65	87.98
5.15	107.53
5.65	78.2
6.15	97.75
6.66	117.3
7.16	97.75
7.66	156.4
8.16	78.2
8.66	156.4
9.16	87.98
9.67	117.3
10.17	136.85
10.67	78.2
11.17	136.85

Grafica:



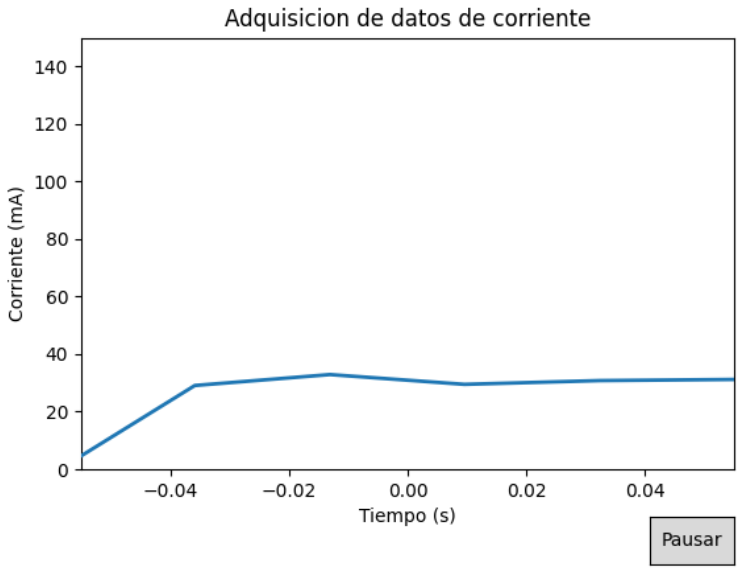
Fuente: elaboración propia.

Figura 7. *Adquisición de datos con el sensor de corriente y voltaje INA219.*



Fuente: elaboración propia.

Figura 7. *Grafica de la Adquisición de datos con el sensor de corriente y voltaje INA219.*



Fuente: elaboración propia.